



BLAZEPHOENIX PROTOCOL · LITEPAPER · FOR EVERYONE

BlazePhoenix

The exchange whose price is computed by the code that spends your money

LITEPAPER — VERSION 2.2

Written by [Mitra](#)

Every number in this document is taken from the technical whitepaper with the same value. Where this text and the whitepaper disagree, the whitepaper wins; where the whitepaper and the source code disagree, the source wins.

Version	2.2
Date	2026-09-06
Author	Mitra
License	CC BY 4.0 (text) · BUSL-1.1 (code)
Web	https://blazephoenix.xyz
DOI	assigned on registration

Contents

1 · What this is, in one page	3
2 · Where the price comes from	4
3 · The quote that is the trade	4
4 · The floor you sign	6
5 · The route	8
6 · Fees	10
7 · How a quote ages	12
8 · Safety: what an attacker can and cannot do	13
9 · What "verified" means here	15
10 · The token, and the second engine	18
11 · What you must trust, and what you need not	19
12 · Reading the receipt	21
13 · How to check any claim yourself	22
Glossary	23
References	25

The exchange whose price is computed by the code that spends your money

Litepaper — Version 2.2

1 · What this is, in one page

You have a wallet — an app that holds the keys to tokens you own on a public blockchain. A **token** is an entry in a ledger that says a certain address owns a certain quantity of a certain thing. A **blockchain** is that ledger, kept by thousands of computers that all agree on it. A **smart contract** is a program stored on the same ledger: anyone can read it, anyone can call it, and nobody can change it after it is published if it was built without a way to change it.

You want to trade one token for another. The tokens are not sitting in a shop. They sit in thousands of **liquidity pools** — small contracts, each holding a stock of two tokens, each willing to swap one for the other at a price its own arithmetic decides. A pool holding 1,000,000 units of one token and 500 of another will quote a different price from the pool next door. There is no single price and no central book.

An **aggregator** is the thing that goes and looks. It reads pools, picks the best combination, and performs your swap. BlazePhoenix-Dex is an aggregator with one distinguishing property, and this document exists to explain it and to show you how to check it:

The price you are shown is computed by the public code that executes your trade, in the frame that executes it. Nobody — the authors included — can change the fee or the floors: they are compiled in.

Five contracts do the work. The **Core** holds the mathematics and holds nothing else. The **Hub** remembers which pools exist. The **Solver** chooses the route. The **Router** is the only one that moves money, and it holds no token balance between transactions. The **Quoter** shows you a preview before you sign. Their sizes, in bytes of published code, are 6,442 · 23,648 · 19,686 · 23,781 · 11,429 — 84,986 bytes in total, all of it readable, none of it replaceable.

What it is not.

- It is not a wallet, and it never holds your keys.
- It is not a custodian. The Router's balance at rest is zero, and no function in it moves your principal on anyone's behalf; the only thing that can sit in it is a mis-send, which a 48-hour public rescue path exists to return.
- It is not deployed yet, in the version this paper describes. The code here is **V2**, and V2 is not on any chain. What is live is **V1**, a different and earlier codebase in its own repository — the same 14,624-byte router on Base, Ethereum, Optimism and Arbitrum, measured by direct query on 2026-09-04. Nothing measured in this document protects the deployed V1 contracts.
- It does not claim to be the cheapest. Working the route out on-chain costs more gas — the fee you pay the network to run a computation — than reading an answer a server prepared, and §11 says what the extra gas buys.

- It does not claim to have eliminated the ordering problem of §7 and §8: the order in which transactions enter a block is decided before any contract code runs, and no contract can change that. This one bounds the damage, twice, and measures the bound.

2 · Where the price comes from

Imagine selling a car in a city with twenty dealers. A good broker visits all twenty, finds the best price, and perhaps splits the sale across two of them. Most on-chain brokers check prices on their own database — possibly stale — and show you a result. You cannot verify that they looked at all twenty, or that they showed you the best one. You can only verify that they have, so far, been honest.

That is not an accusation. It is an architecture. Every aggregator surveyed for the technical whitepaper works the same way: a service off the chain computes the route, and the contract on the chain executes the route it is handed. The contract does not choose; it checks. That is sound engineering — it is why those systems are fast and cheap — and it has one consequence rarely stated plainly: **the strongest claim such a system can make is that its operator is honest.**

The gap is not usually fraud. It is drift, from three ordinary sources. The quote was computed against pool balances that moved before your transaction landed. The search was only as exhaustive as the operator's budget. And every correction between the quote and the fill — a safety margin here, a fee there — was applied by code you did not read, to a number you cannot reproduce. None of that requires bad intent. All of it is invisible.

A worked example, with the smallest arithmetic in this document. A pool holds 1,000,000 USDC and 500 WETH. Its rule is the oldest one in the field, called **constant product**: the two stocks multiplied together may never shrink. You sell 10,000 USDC into it, and the pool keeps 0.30 % of your input as its own fee. You receive **4.935790 WETH**.

At the price the pool showed a moment before you traded — 0.0005 WETH per USDC — the same 10,000 USDC would have fetched 5 WETH. You are 1.28 % below that. Of the 1.28 %, exactly 0.30 % is the pool's fee. The rest is your own trade moving the price against you as it goes through: **slippage**, and it is arithmetic, not misconduct. The more you push through one pool, the worse each further token pays.

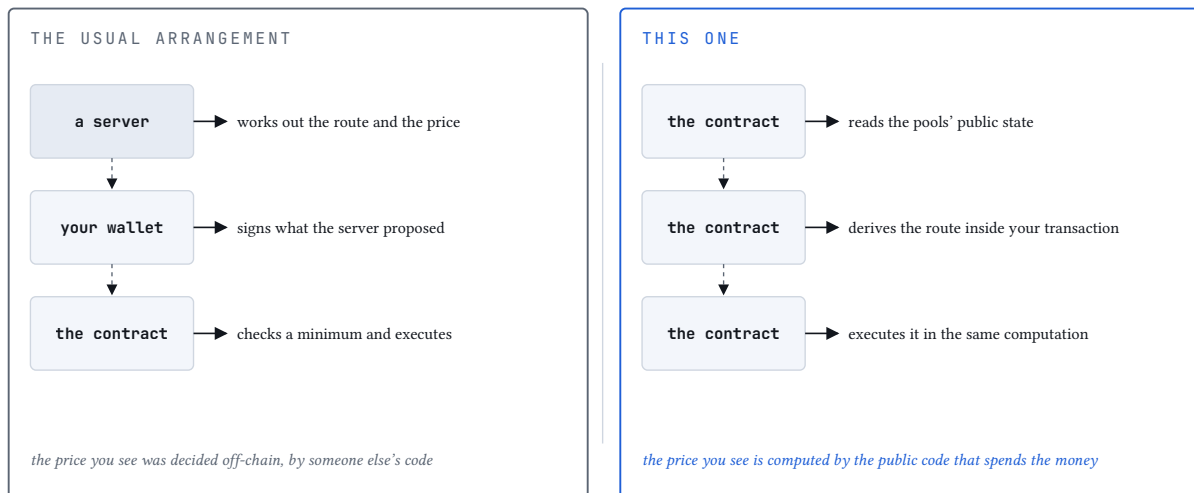
Two questions follow, and the rest of this document answers them. Who computed the 4.935790 — a server, or the contract that will spend your money? And what stops the number you are actually paid from being 4.1 instead, when the world moves between your signature and your fill?

3 · The quote that is the trade

The thing that chooses is the thing that trades.

BlazePhoenix has an entry point that takes six values: the two tokens, the amount, your minimum, who receives the output, and a deadline. It then works out the route *inside the transaction that executes it*, from pool balances that are public state — which venues, how much to each, in what order, all derived in the same frame by the same code that is about to move the tokens.

Because the route came from public state by public code, anyone can re-derive it; because the number that binds at settlement is re-derived by the same dispatcher, in the same frame, from the same live state, the quote and the execution are not two artefacts to reconcile but one computation. What separates a preview from a fill is time, not a second model.



LITE. 1 Who computes your price: the usual arrangement, where a server decides the route and the contract checks it, against this one, where the contract derives the route from public pool state inside the transaction that spends the money.

The machinery has five named parts. Each gets its plain meaning here and its precise definition in the whitepaper.

- The **Meta-Equation** is the whole aggregator written as one maximisation: among the routes it can build, take the one that returns the most after the fee, provided it clears its own floor and touches nothing inadmissible. It is a product, so a single failing factor makes a route worth exactly zero rather than merely worth less — no soft penalties, no near-miss routes.
- The **Eightfold Dispatcher** Ω is one function that knows how to price every kind of pool. The name is historical; six kinds are live today, collapsing into four mathematical families — constant product, concentrated liquidity, the Solidly quartic, and the Uniswap V4 singleton. Adding a venue means assigning it a kind, not writing new pricing code. Where a curve is too intricate to reproduce safely, the dispatcher asks the pool's own code for the number rather than copying its mathematics, because a copy that drifts is a mispriced quote.
- The **Deterministic Derivation** $\mathcal{D}$ is how a pool is found. Where a venue family's deployments are reproducible, a pool's address is *computed* from three public inputs by fixed arithmetic — a small theorem anyone can recheck — instead of fetched from a directory. Where the arithmetic does not apply the protocol asks, and since September 2026 an asked pool must answer with its own two token addresses, proving it really trades the pair, before it is listed. An address that is computed can be verified by anyone; an address that is fetched can only be trusted.

- The **Vitality Field Ψ** scores how much a pool deserves to be asked: how deep, how recently traded, whether it sits on a heavily used pair. It ranks candidates and keeps the best eight. Its limit is the most important sentence about it: **Ψ can hide a venue; it can never misprice one.** No quote, no floor and no allocation reads it.
- The **Monoslot** is that score's storage: a pool's entire routing state packed into one 256-bit word — activity, depth class, fee, kind, flags, timestamps — read in one go. Notice what the word does not contain: a price. The memory can bias which pools are worth asking and is structurally incapable of pricing a trade.

Behind all five sits the discipline the whitepaper names **Metrological Design**: build the contract as an instrument. Measure every quantity that can be measured; where a number cannot be, shape the function so that lying about it can only hurt the liar. Data supplied by a stranger — a route, a claimed depth, a declared fee — are not beliefs but places to point the instrument, which the contract then reads for itself.

The registry that remembers pools is **self-healing**: it learns liquidity by trading it. Every executed leg tells the Hub what it just used, and the Hub records it — one packed word rewritten, riding inside a transaction you were already sending. There is no indexer to run and no moment at which the protocol goes blind because a server is down.

What that costs, in one number. Solving a route in-frame for the first time on a pair, sweeping four registered venue directories, measured **169,093 gas**; doing it again once the registry has learned those pools, with the sweep skipped, measured **132,691 gas** — a saving of 36,402 gas, or 21 %. If you would rather not pay for the search at all, three further entrances accept a route you computed yourself, for free, through the Quoter's preview, and the same measurement, floors and settlement rules apply to all four.

4 · The floor you sign

There is one number in a swap that no contract can work out on your behalf: how much less than the preview you are willing to accept. It is called the **minimum output**, it is yours, and here it is mandatory. A swap that names zero does not execute — every entrance refuses it before a token moves, with a refusal code the contract publishes, and there is no compatibility flag to switch that off. A guard that may be absent is not a guard.

Underneath your number the protocol puts one of its own, because not every route that *can* execute *should*: a route returning 60 % of fair value is worse than no trade. That second number is the **Iron-Law floor Φ** — the protocol's own retention floor, re-derived inside the executing transaction from what actually happened, and applied to the last hop's quote at the amount that actually arrived there.

Worked example, with round numbers. You are selling token A for token B by way of WETH. Two hops. In the first hop the route sends 600 A into one pool and 400 A into another; in the second hop, the WETH those two produced goes through a single pool. Inside the transaction the Router measures how much each pool's price moved: 100, 60 and 40 basis points. A **basis point (bps)** is one hundredth of one

per cent, so those are 1.00 %, 0.60 % and 0.40 %. Weighted by how much each leg carried, the route's measured impact is 80 bps. The final hop's quote, computed in the same frame at the WETH that really arrived, is 1,000.000 B.

1. The floor starts at 96 %.
2. It loosens by one basis point for every basis point of measured impact: 96 % – 0.80 % .
3. It loosens further because the trade was genuinely split rather than concentrated. Splitting 600/400 counts as 1.923 effective legs, which earns 184 bps of loosening — and a leg carrying a vanishing share earns a vanishing amount, so declaring a token of dust as a second leg buys nothing.
4. That leaves **93.36 %**. The protocol's own floor is 93.36 % of 1,000.000 B = **933.600 B**.
5. You set your minimum at 990 B. The binding number is the larger of the two: **990 B, yours**, as it should be on an honest route.

A delivery of 998.400 B settles. A delivery of 985 B is refused — the transaction reverts whole, every pool's swap included, and you keep your tokens. And had you set your minimum at 900 B instead, the protocol's 933.600 B would have bound: a fill at 930 B is refused even though it clears the number you asked for.

The floor never falls below 80 %, whatever the inputs. That is the whole shape of the rule: start at 96 %, loosen one-for-one with the impact the trade really caused and 200 bps per genuinely extra leg, and stop at 80 %.

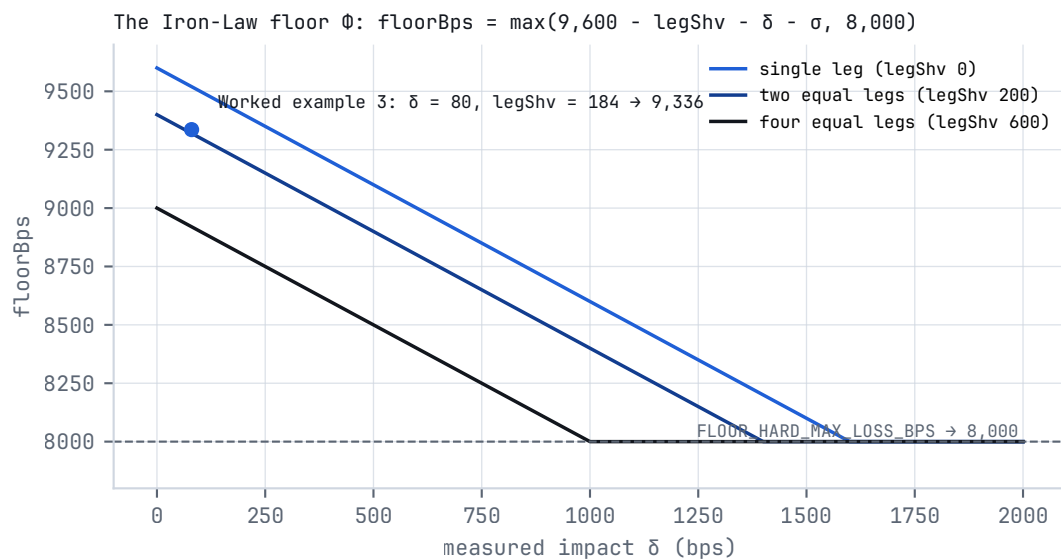


FIG. 7 The Iron-Law floor: retention against measured impact for a single leg, two equal legs and four equal legs; the 80 % hard clamp; the point of Worked example 3 marked.

Two properties make this an enforcement rather than a decoration.

Numbers supplied by the caller can tighten protection and can never loosen it. The route handed to the Router carries advisory fields — a quoted total, a floor the planner attested — and the Router trusts neither: it measures, recomputes, and takes the *largest* of your minimum, the attested floor and its own recomputed floor. A route that understates its own expectation does not thereby switch its own floor off; an attested bound covering less than half of the measured quote is lifted to the measured quote.

Every leg has its own floor as well as the route. Each individual leg must return at least 80 % of what it was bound to, or the whole swap reverts, so a single manipulated pool fails the transaction immediately instead of hiding its loss inside a healthy total. And because a per-leg rule is local while an attacker holding one leg of many is not, each hop may lose at most what one *average* attested leg could legitimately lose — a rule that collapses exactly onto the per-leg one when a hop has a single leg.



The binding number is whichever floor is highest — here yours (990 B); the protocol's floor (933.600 B) is the guarantee under it.

LITE. 2 The three numbers under one trade, on one scale in the output token: the in-frame quote (1,000.000 B), the protocol's floor (933.600 B), the minimum you signed (990 B), and the amount delivered (998.400 B); the binding number is whichever floor is highest.

The one habit worth keeping from this document: set the minimum yourself, deliberately, every time. Derive it from your own expectation of the price, not from a field in a preview. It is the only protection in this system whose quality depends on nobody's competence but yours.

5 · The route

The reason to split one order across several pools is arithmetic, not cleverness. Because your own input sits in the denominator of a pool's pricing rule, the more you push through one pool, the worse each further token pays. Sending half the order to a second pool moves both pools less.

Worked example, one order of 100. Two pools trade the same pair, both charging 0.30 %. Pool A holds 1,000 a side; pool B holds 4,000 a side.

ROUTE	YOU RECEIVE
All 100 into A	90.661
All 100 into B	97.275
20 into A, 80 into B	19.550 + 78.201 = 97.751

The 20/80 split is not a guess. Weights are proportional to *measured* depth against the deepest candidate in the set: A weighs a quarter of B, so A takes 20 % of the order. The split beats the best single pool by 0.476 tokens — 48.9 bps of the order — and every row is checkable with a pocket calculator. This rule is deliberately simpler than the theoretical optimum for the problem [1]: it needs no iteration and no model of each venue's curvature, which makes it cheap in gas and hard to game.

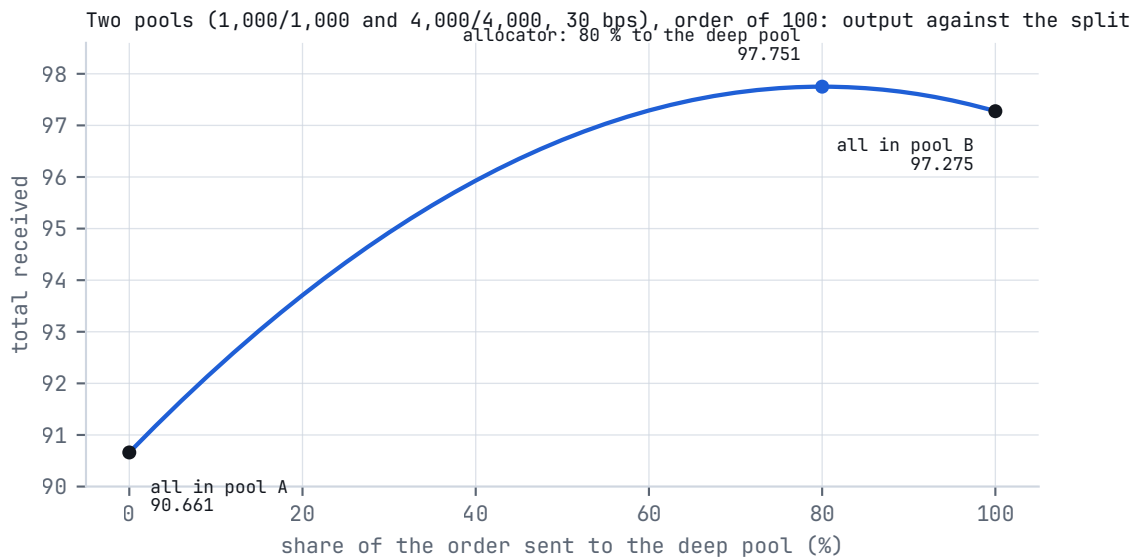


FIG. 6 Output against the share sent to the deep pool for the two-pool example: a concave curve with its maximum near 80 %, the allocator's depth-proportional point marked.

An extra leg costs real gas, so a split has to earn its second leg. The gate is a threshold: keep the split only if it beats the best single pool by at least 25 parts per million. In this example the threshold is 97.278 and the split returns 97.751, so it stands. The number 25 is itself a measurement: the true break-even between an extra leg's gas and its improvement measured 3 parts per million on Base — 320 bytes of extra calldata at 105.5 gas per byte — and the gate sits about eight times above it. Its history is on the record in the whitepaper: 20 bps, then 5, then 0.25, before anyone measured what it was supposed to be protecting.

The rest of the route machinery is a funnel, and each stage has a stated limit.

- **Candidates.** For each pair, the remembered pools and — unless the registry has been touched recently enough by curated entries — the derived ones, ranked by Ψ and cut to eight. Eight is deliberately wider than any hop's leg budget, so a deep pool listed behind several thin ones is seen rather than starved by list order.
- **Probe.** Each candidate is priced once at a hundredth of the order, capturing its rate and its depth in one read. A small-but-healthy pool has a correct rate at that size and merely a poor full-size output; a stale-priced pool has a wrong rate at any size. The small probe separates the two.
- **Believability.** Candidates whose rate departs by more than $\pm 5\%$ from the centre of the set are dropped before anything is sent. The centre is the *depth-weighted median*: to move it, an attacker must out-weigh half the pair's real liquidity. It is deliberately not anchored on how many tokens a pool holds, because anyone can send tokens to a pool and take them back — an outside researcher demonstrated exactly that capture, and the anchor was replaced.
- **Capacity and shape.** A concentrated-liquidity pool's formula can promise more than the pool has ever held — 117 times more in one observed case — so such a leg's promise is capped at 30 % of what

the pool measurably holds of the output token. And every dimension is bounded: at most three hops, at most four legs in a hop when the Solver plans and five when you supply the route. A pair with no admissible route is refused rather than fabricated.

One omission is deliberate: a planning haircut that shaved every multi-leg route by 5 bps "as insurance" was retired when the hazard it insured against measured 2 bps at worst — 27 times smaller than the premium every honest route was paying. The risk now sits on the floors, which are checked against what really arrived.

The reader's rule of thumb survives all of this unchanged: compare the improvement the router claims against the gas your wallet shows, before you confirm.

6 · Fees

The protocol takes **28 basis points** — **0.28 %** — **once**, from one measured amount, in a coin the treasury wants to hold, and the contract counts that it did.

The rate is a compile-time constant with no setter anywhere in the published code. It is split 30 % to a first treasury and 70 % to a second, and that split has no setter either. No key can raise the rate against you and no key can lower it for a favoured integrator: the fee question was answered when the code was compiled and removed from the list of things a key can do.

Where it is charged. The Router scans your route for the first hop whose input is a **bridge coin** — one of at most three widely-held tokens, WETH and USDC in practice, that most routes pass through — charges 28 bps of that hop's *measured* input, and charges nothing else on the route. This is the **anchored regime**, and the reason is plain: the treasury receives a liquid coin instead of dust of whatever tail token you happened to be buying. A one-hop route whose destination is a bridge coin is charged on the output instead, after the floor check has already validated the swap.

Worked example, selling 10,000 A.

Two hops, $A \rightarrow WETH \rightarrow B$. The first hop is not charged, because A is not a bridge coin. Its pools pay out 5.000 WETH, measured as the Router's own balance change. The fee is 0.28 % of 5.000 = **0.014 WETH**, split 0.0042 and 0.0098, and the second hop spends 4.986 WETH into B. One fee event. Nothing is taken from B.

One hop, $A \rightarrow WETH$. Nothing is charged on the input. The pools deliver 5.000 WETH gross; the floor is checked on 5.000; then 0.014 WETH is taken and **4.986 WETH** is delivered, and your minimum is compared against the 4.986 you actually receive.

A route through no bridge coin at all, $A \rightarrow C \rightarrow B$. No hop's input is a bridge coin, so there is no single place to anchor, and every hop pays 28 bps of its own measured input: 28 A at the first hop, then 5.6 C at the second, an effective rate of 55.9 bps. This is the **exhaustion regime**, and it is a deliberate rule rather than an oversight. Charging such a route only once, at the first hop, was tried inside the test suite on 2026-09-05 and immediately reopened the escape the rule exists to close — a worthless first hop carries the fee onto dust and the real hop pays nothing — and five pinned tests refused the change within the same run. There is no index at which to insert dust that escapes every hop. The Solver builds every multi-hop

route through bridge coins, so a route this protocol plans for you always pays exactly once; the exhaustion regime is what you meet if you route around the registry yourself, and the preview models it, so what you are shown and what you are charged agree either way.

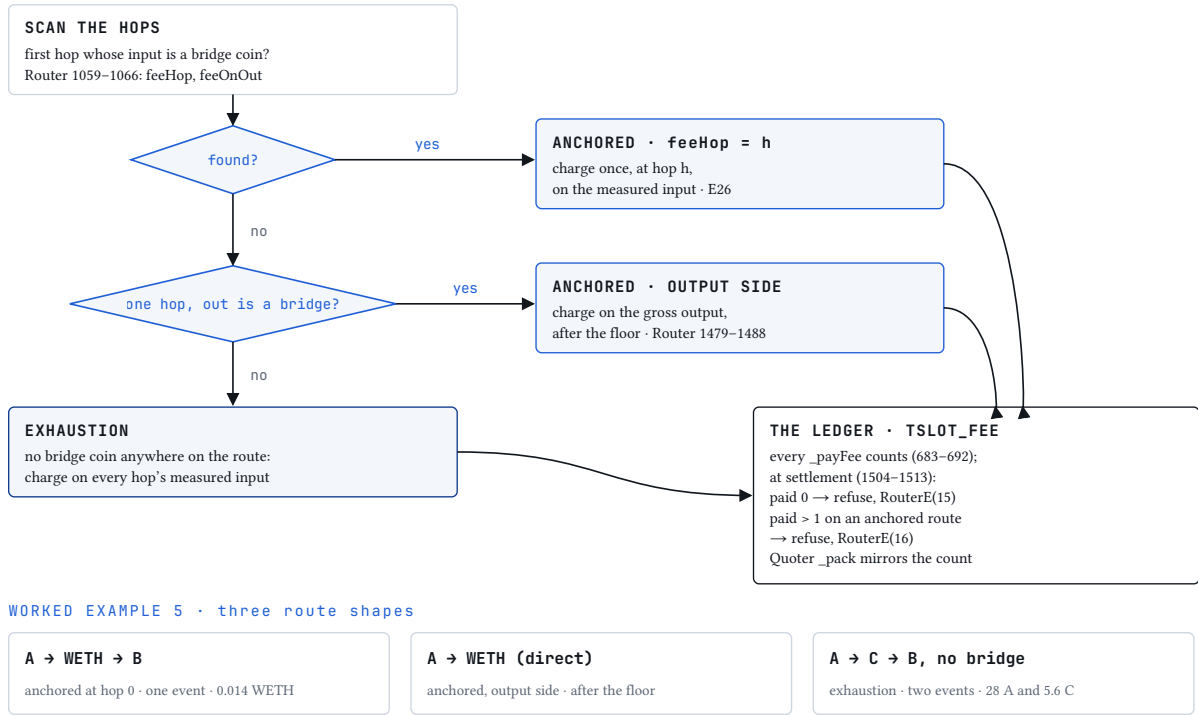


FIG. 9 The two fee regimes as the Router decides them: scan the hops for the first bridge-coin input; anchored routes pay once at that hop (or on the output of a direct route into a bridge coin); routes with no bridged input pay once per hop; the ledger counts and refuses zero, and refuses two on an anchored route.

The Surplus Rule is the alignment argument, stated as the code implements it: the take is 28 bps of one measured base, and *no term anywhere in settlement scales with the difference between what was quoted and what was delivered*. Where the base is an input — every multi-hop route, and every one-hop route not ending in a bridge coin — the output is untouched, so everything the pools deliver above the quote reaches you in full and the fee cannot rise because your fill came in favourable. Where the base is the output, the fee is 28 bps of the gross delivery and nothing more. The axis on which an aggregator is most tempted to skim is the gap between quote and fill; this design gives that gap no fee term at all, so the headline rate is the true and only rate.

Two further protections sit under the rule. The base is never read from your route: the number your route claims as its total output is read by nothing, and the fee is computed from a balance the Router measured for itself. And the Router counts its own fee payments during the swap: a settlement that paid the protocol nothing is refused, and so is a second payment on an anchored route — with the limit stated as the whitepaper states it, that no test can make either counter fire, because the rule in front of them leaves no path that settles without paying. They are the contract refusing at run time what the tests refuse at review time.

7 · How a quote ages

A quote is a statement about the past, and your trade happens in the future. Between the moment a preview is computed and the moment the transaction is included in a block, other people trade through the same pools. The honest question is not whether that gap exists. It is how much it costs, and what happens at the far end of it.

The experiment takes a quote exactly the way an integrator does — one call returns both the preview and the transaction data — then lets the world move: zero to three trades by somebody else through the same pools, each up to 3 % of the shallow side, **all of them in your direction, so every one of them hurts**. Then zero to ten seconds pass, and the unchanged transaction data is executed. 240 samples, every outcome classified.

DRIFT BETWEEN QUOTE AND EXECUTION	SAMPLES	SETTLED	REFUSED BY THE FLOOR	DELIVERED ÷ PREDICTED, MEAN	WORST
none	64	64	0	10,000 bps	10,000
1 – 100 bps (up to 1 %)	43	43	0	9,952 bps	9,803
100 – 300 bps (1 – 3 %)	133	102	31	9,966 bps	9,653

DELAY BETWEEN QUOTE AND EXECUTION	SAMPLES	SETTLED	REFUSED	DELIVERED ÷ PREDICTED, MEAN	WORST
0 s	17	15	2	9,989 bps	9,857
1 – 5 s	99	89	10	9,959 bps	9,653
6 – 10 s	124	105	19	9,983 bps	9,659

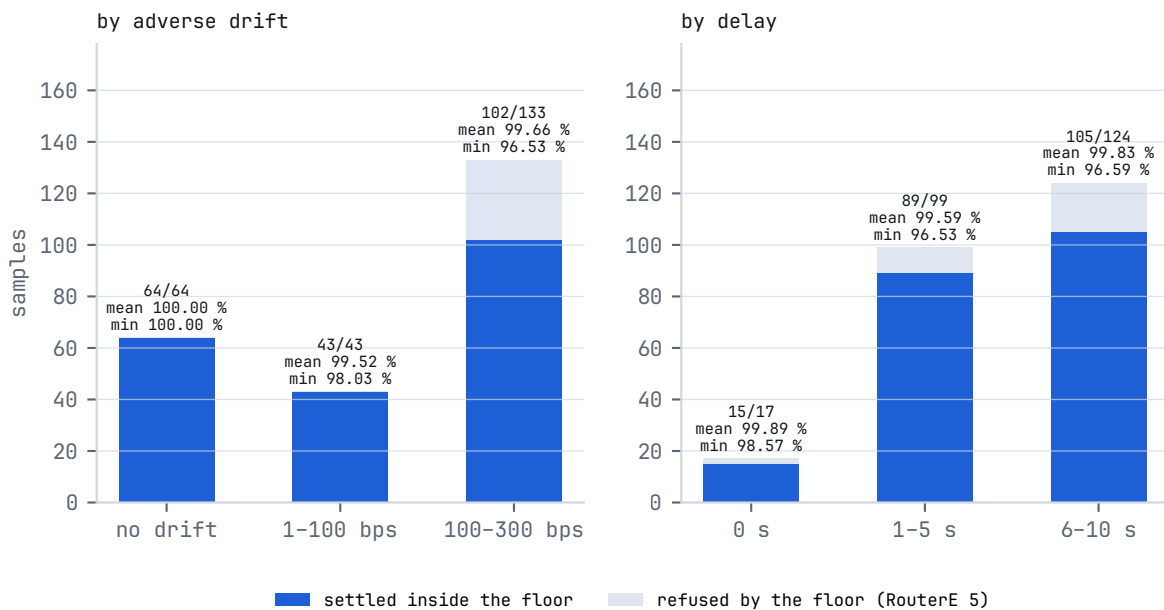


FIG. 8 How a quote ages: settled and refused samples by drift bucket, with the delivered/predicted ratio of every settled sample; the same 240 samples bucketed by delay show no trend.

Read plainly, in four sentences. **Time does not move a quote; other people's trades do** — the delay table has no trend and the drift table has a strong one. **A quote against a pool nobody disturbed delivers exactly what it predicted, 64 times out of 64, at every delay tested. Up to 1 % of adverse movement, every quote still fills, within 2 % of its prediction. Between 1 % and 3 %, the floor refuses roughly one in four rather than fill below what the preview attested, and the ones that do fill land within 3.5 % of the prediction.** There is no third outcome, and after the deadline had passed 20 of 20 were refused with the deadline's own code and none settled.

The same experiment on real Base liquidity, with the protocol deployed on a fork of the live chain at the current block, selling 1,000 USDC for WETH: executed 0, 3, 6 and 10 seconds later with nothing else moving, the quote delivered exactly its prediction all four times. Then, with somebody else's trade pushed through the same route first and the stale transaction data executed 10 seconds afterwards:

TRADED AHEAD OF YOU	YOU RECEIVED, AS A SHARE OF THE PREDICTION
10,000 USDC	9,999 bps
50,000 USDC	9,998 bps
200,000 USDC	9,994 bps
1,000,000 USDC	9,974 bps
5,000,000 USDC	10,000 bps

A million dollars traded ahead of your thousand costs your thousand a quarter of a percent. The five-million-dollar trade cost nothing at all, because at that size it routed through pools your thousand-dollar route does not touch.

The limit, in the same breath as the result: the 240-sample distribution is over simulated pools and the Base run is one pair at one block. The distribution gives shape, the live run gives reach, and neither is a promise about a pair you have not tested.

8 · Safety: what an attacker can and cannot do

Attacks on an exchange front-end come from two directions: the venues it trades through, and the person who sees your transaction before it is included. The protocol treats them differently, because one can be closed and the other can only be bounded.

A hostile venue. A pool is somebody else's contract. It can pay nothing, pay half, answer a price query with 64 kilobytes of garbage, burn every unit of gas it is given, demand payment twice inside one swap, or call back into the Router before it has paid. Ten such misbehaviours were built and crossed with both ways of entering the protocol — a route you supply, and a route it solves — for 20 cells in all. **Every one of the 20 either settled correctly, with the delivered amount equal to the recipient's measured balance change and nothing left behind on the Router, or was refused with a code belonging to this protocol.** No third outcome occurred in any cell.

Three rows show the shape of the defence rather than its result. A venue that burns all the gas on a *price read* costs a bounded amount and is dropped, because every read the Core makes to an outside contract is capped at 100,000 gas. A venue that burns all the gas inside the *swap itself* reverts the whole transaction and leaves your balance untouched. And the last row changed the code: on its first run, discovery listed a pool that a properly admitted directory had answered with, on tokens that were not the pair — the executor refused it at the seam that pays, so no funds were at risk, but an impostor had won a slice of the split. An asked pool now proves its own two tokens before discovery will list it.

A hostile ordering. Somebody who sees your pending transaction can trade ahead of it, let you fill at the worse price, and trade back. This is a sandwich [7][8], and it cannot be prevented from inside a contract, because ordering is decided before the code runs; a protocol claiming to have eliminated it has either moved your trade off the public queue, trusting whoever now holds it, or is describing something narrower than it sounds. What this protocol does is bound it, twice and independently: your minimum caps the damage at the number you chose, and the Iron-Law floor caps it again on the protocol's own account, re-derived from what the Router measured rather than from anything the attacker supplied.

The bound was measured from the attacker's side. The victim trades 10,000 into a pool holding 1,000,000 a side — 1 % of its depth — with the route and floor fixed at quote time, exactly as a pending transaction has them.

ATTACKER MOVES, AS A SHARE OF THE POOL'S DEPTH	VIC- TIM	VICTIM'S LOSS AGAINST THE QUOTE	ATTACKER'S ROUND TRIP
0.1 %	settles	0.47 %	+ 13.9
0.5 %	settles	1.26 %	+ 68.9
1 %	settles	2.22 %	+ 136.7
2 %	settles	4.12 %	+ 268.7
3 % and beyond	re- fused	0	− 174.5 ... − 544.9

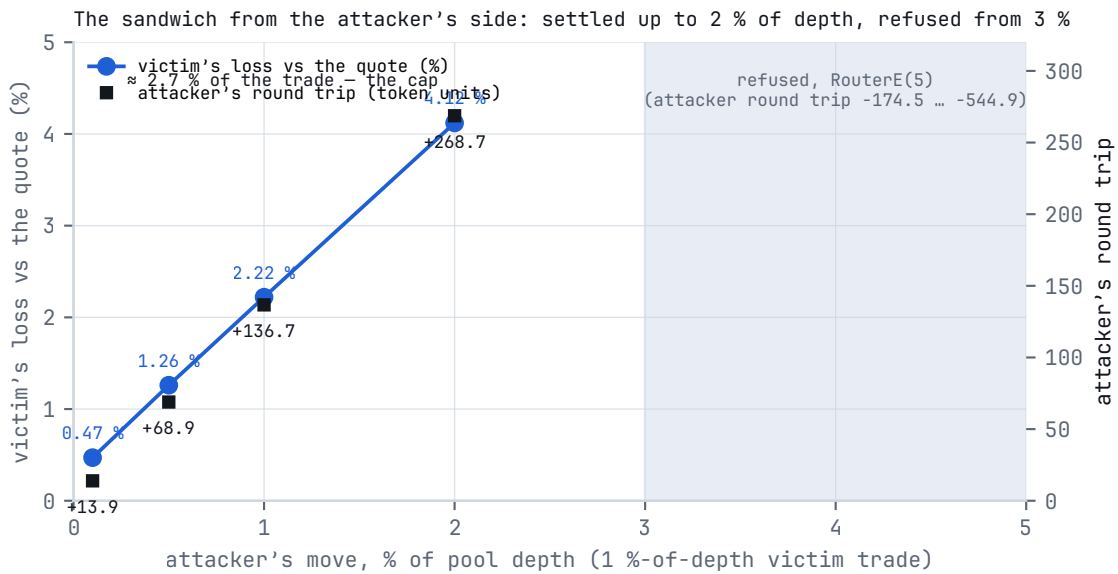


FIG. 12 The sandwich curve from the attacker's side: the victim's loss against the quote and the attacker's round trip, by fraction of depth moved ahead of the victim; the refusal edge near 3 %, closed upward.

Two things are asserted at every point on that curve. A victim who settles never receives less than the floor attested when the quote was taken, so the loss is bounded by the distance between the attested quote and the attested floor. And the refusal region is closed upward: past the edge, *every* larger manipulation is refused and the attacker is left holding the price they moved, which is why the last column turns negative. The number worth quoting is the last settled row: on a trade of 1 % of a pool's depth, the floor caps what a sandwich can take at about 2.7 % of the trade, and turns the attacker's trade into a loss the moment it would take more. A quieter deterrent rides alongside: when the route is derived inside the executing transaction, there is no pre-published path to study, because the path does not exist until the block that executes it.

A hostile front end. On every aggregator ever built, the interface proposes the minimum, and a careless one proposes it badly; no contract can know what you would have chosen. What a contract can do is make the proposal permanent and public: every settlement writes a record on-chain naming the quote used, the amount delivered and the floor applied (§12), so an interface that consistently proposes bad minimums writes its own indictment into the log, where anyone can audit it. The defence that completes the mechanism is a habit: look at the minimum before you sign.

9 · What "verified" means here

Most projects say "audited" and "tested" and stop. Those words carry no number, so this section gives the numbers and, immediately afterwards, what they do not establish.

The suite. 1,478 test declarations across 218 files. On the settings the release build uses, **1,319 passed, 0 failed, 1 skipped, of 1,320** — the skip and the denominator both printed, because a pass rate without its denominator is a decoration. A separate lane runs 39 suites against real liquidity on forks of five chains: **119 of 119**.

The mutation guard. A green test suite proves that the tests pass. It does not prove that they *could have failed*. The instrument that asks the second question is called mutation testing, and it is old — introduced in 1978 [12]. You take one exact line of the source, break it deliberately, and require a specific named test to go red. If nothing goes red, the line was never really guarded.

This repository holds **203 hand-written mutants, each paired with the single test that must fail once that line is altered**: a guard deleted, a comparison flipped at the exact bound that decides a refusal, an authorisation widened, an error code swapped for its neighbour's. All **203 of 203 are killed**. Three properties make it a guard rather than a score: the paired test is run green on the unbroken code first; a mutation the compiler optimises away — producing byte-identical code — is reported as *inert*, never as killed; and a check verifies that every mutant still points at exactly one line. Its limit is printed with it: a saturated score is a floor on what has been considered, never a ceiling on what exists.

Mutants aimed at the properties, not the lines. Forty properties are asserted to hold in *every* state a random sequence of actions can reach — the Router holds nothing, the fee is charged on exactly one side, a pool in the registry really trades the pair it claims. Until 2026-09-05 all forty were green and nobody had asked whether any *could* go red; a campaign that never reaches the state a property protects certifies it over an empty universe, and looks identical from outside to one that reaches it. So fifteen source mutations were run against all forty: **eleven were noticed on the first measurement, and twenty distinct properties went red at least once**. Three of the four survivors were closed the same day by giving a campaign the action it lacked, each verified red without its guard first. **Fourteen of fifteen are now noticed**, and the fifteenth survives by design: a regression sentinel no campaign can reach, watched by two unit mutants instead. Two guards with no watcher of any kind was the number this measurement existed to print, and it is now zero.

Covering arrays. Coverage tells you which lines ran; mutation tells you whether a line was guarded. Neither tells you which *situations* were set up before the call. Ten such factors are enumerated — venue family, hop count, legs per hop, whether the input is a bridge coin, whether the intermediate is, whether the token taxes its own transfers, the token's decimal convention, which entrance was used, whether administrative control has been surrendered, and whether the pair's sixteen registry seats are full. That is **5,184 combinations**, far too many to run. A covering array is the standard construction for exactly this [14]: a small set of rows in which every *pair* of factor values, or every *triple*, appears at least once.

STRENGTH	ROWS	COMBINATIONS HELD	SET- TLED	REFUSED WITH OUR OWN CODE	NOT CON- STRUCTIBLE	THIRD WAY
every pair	63	258 of 258	53	4	6	0
every triple	168	1,636 of 1,636	158	10	0	0

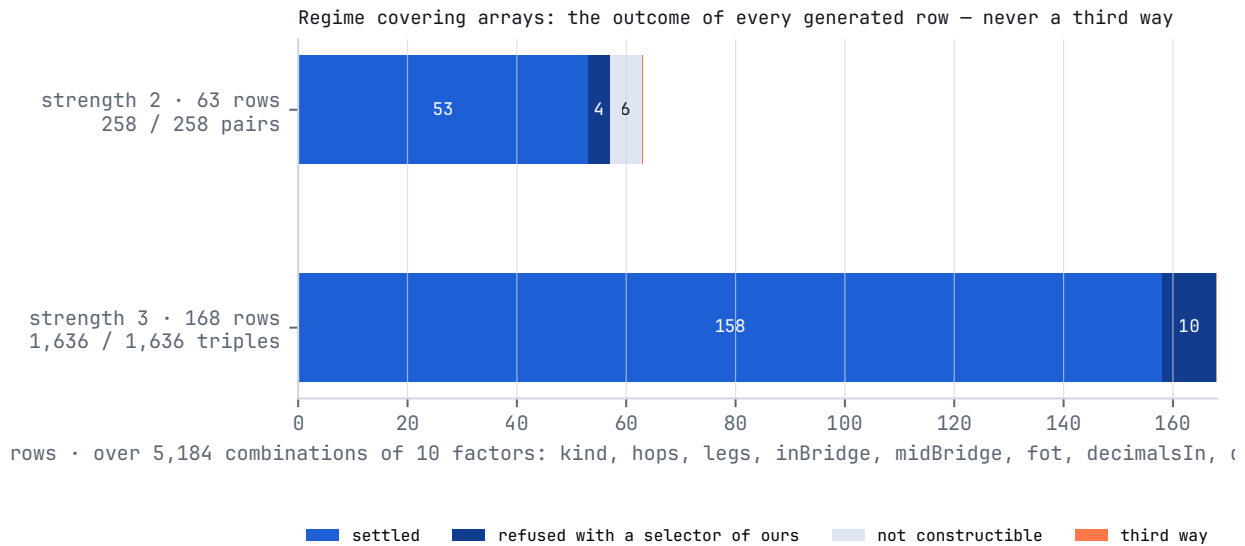


FIG. 14 Regime covering arrays: outcomes of every generated row at strength 2 (63 rows) and strength 3 (168 rows) — settled, refused with a selector of ours, not constructible, third way — over 5,184 combinations of ten factors.

Every row is judged by one rule: either the swap settles — delivered amount equal to the recipient's measured balance change, at least the floor the contract published, nothing left behind — or it is refused with a code belonging to this protocol. Anything else is a *third way*, and a third way fails the row. Zero occurred in either array. The six rows that cannot be built are printed by name and count against the denominator rather than being quietly dropped, as are the venue families the arrays do not cover.

Three more instruments, each answering a question the others cannot.

- *Canonical oracles.* Every simulated pool in the suite prices with the protocol's own formulas, so a defect in a formula is invisible to every test that uses them: the oracle is the object. Three independent implementations were therefore written from the venues' published specifications [3][4] and the protocol's mathematics fuzzed against them, 5,000 runs each — constant product exact to the smallest unit, the Solidly stable curve within 4 units, concentrated liquidity below one unit for every pool in the ordinary range. Their limit: they were written from specifications, not from the venues' compiled code.
- *Metamorphic relations.* Where no oracle independent of the formula exists, you can still ask how the output must *move* when the input moves [11]: fourteen such relations over the quote mathematics and five over the chosen plan — more in never returns less out, splitting an order across the same pool never gains, a higher fee never pays more, adding a pool inside the price band never lowers the plan.
- *N-version.* The suite executes one compiled binary, and every other compiler setting is a different program built from the same source [13]. The quote mathematics is compiled under two further settings and all three binaries asserted equal on fuzzed inputs, 3,000 runs each, the lane first proving the alternate code really differs so it cannot pass by comparing a program with itself. The compiler was not the finding: on its first full-domain run the lane made the *reference* implementation revert, exposing a guard in the stable curve whose own arithmetic overflowed on a pool holding less than one unit of a token. Not a wrong number — a revert inside a quote, which would have unwound the planner's whole answer for that pair. It was repaired first with a failing test, and the repair is 35 bytes *smaller* than the code it replaced.

What none of this establishes, stated as carefully as the numbers:

- **No probability of correctness.** The literature is explicit that testing cannot produce one [15], and none is stated here.
- **Mutation adequacy is adequacy against this register.** The register is hand-curated.
- **Threat and situation coverage are floors on what has been considered.** Anything nobody has named is outside the denominator by construction.
- **The ageing distribution of \$7 is over simulated pools.** Live Base gives reach, not a distribution.
- **This repository is V2, and V2 is not deployed.**

One line belongs here because it is the discipline rather than the result. Seven of the instruments above were wrong on their first run, and each was corrected against a defect already known by another route *before* its number was printed: an instrument is worth nothing until it finds the instance already known.

10 · The token, and the second engine

BZPX is a plain token with a fixed supply: **one billion units, for ever**, eighteen decimal places, no way to mint more, no tax on transfer, no rebasing, no transfer hooks, no pause switch and no blacklist — six properties checked against the deployed token's own code when it is wired up. Fully diluted supply is known from the first day; scarcity is a property of the contract, not a policy anyone administers.

ALLOCATION	TOKENS	SHARE	NOTE
Market liquidity	550,000,000	55.0 %	the float the aggregator routes through
Staking emission	180,000,000	18.0 %	the staking engine's entire budget
Team and operations	103,000,000	10.3 %	the only locked insider tranche
Partners and ecosystem	60,000,000	6.0 %	integrations, venue partnerships
Airdrop	50,000,000	5.0 %	early users and community
Security research	50,000,000	5.0 %	the bounty pool below
Marketing	7,000,000	0.7 %	launch awareness

At the time of writing the token has no provisioned liquidity and no generation event. On-chain volume of zero is the expected state of this stage, not a signal about the code. When the token contract publishes, its own on-chain record — not this table — is the canonical version.

The second engine, by reference. A staking engine for BZPX — a single-asset vault in which the collateral, the reward and the accounting are one token, and solvency is a revert condition on every value-moving transaction — is specified in full in its own paper [23], on its own track, and is under construction. This document deliberately does not summarise it: its emission mechanics, its accounting identities and its own edges belong there and are stated there. Two facts belong here because the aggregator's record depends on them. The 180,000,000-BZPX line in the table above is that engine's entire budget, hard-capped in its funding path. And the research pool below is one pool shared between the two repositories.

What hostile reading pays. Before this project had a research programme worth the name, independent researchers read the source hostilely — unasked, with nothing promised — and disclosed what they saw privately. **Nineteen researchers have been credited, and no report has reached the Critical class.** The pool is 50,000,000 BZPX, 5 % of the fixed supply, with awards from up to 125,000 BZPX for a demonstrated low-severity impact to 2,500,000 – 7,500,000 for a Critical, paid in BZPX — quantity and schedule promised, market value not — with payouts beginning after October 2026 and reports triaged now. One clause is deliberate: a report need not break a conservation law to be Critical, because a redistribution that systematically pays the wrong party while the books still balance is priced like a theft. That is the class only hostile readers catch. Report privately to contact@blazephoenix.xyz.

11 · What you must trust, and what you need not

Almost every protocol calls itself trustless. Very few can enumerate their exceptions. Here is the full list, each power stated with the line that bounds it.

Administrative keys exist today, and this is exactly what they reach.

- *The Router's control key* can redirect where the fee goes, pause swaps, repoint the two canonical outside contracts its native-currency and signature doors call — which makes those two doors only as trustworthy as the addresses set — and recover mis-sent tokens behind a 48-hour public delay announced on-chain when it is queued. It cannot move the fee rate, the split or the floors, which are compile-time constants with no setters; and it cannot take your principal, because there is no proxy, no upgrade path, no self-destruct, no standing approval of your funds to the Router, and nothing held at rest. The functions that would do it are not disabled. They are absent.
- *The Hub's control key* can list a venue or a hook, seed pools, pause the registry's learning, and repoint the Hub's view of the other contracts. It cannot misprice a fill, because nothing that prices a trade reads the registry: the sharpest thing a hostile listing reaches is the gap between a quote and *your minimum*. Principal is out of reach; your slippage tolerance is not, which is one more reason to set the minimum tightly.
- *The treasuries* are fixed at construction and the 30/70 split has no setter.
- *The canonical outside contracts* the doors depend on — the signature-transfer contract for one door [19], the chain's wrapped-native token for another, the singleton pool manager for Uniswap V4 legs [5] — are each called only through their own door or leg, and the Router prices every leg off its own measured balance change, so a dishonest counterparty collapses the measured output and your own bound refuses the swap.

The One-Way Door is how that list ends. Both the Hub and the Router expose a function that surrenders control irreversibly, in code rather than by promise: the flag is written *true* at exactly one place in each contract and written *false* at none, so no function in the published code can reopen it. Each refuses to be called while paused, because a paused-then-renounced protocol would be a terminal state nobody wants and nobody can leave.

What closes and what stays open is enumerated rather than summarised — the part worth checking whenever anyone says "renounced" about any protocol. On the Router everything privileged dies: the treasuries, the wiring and the pause flag freeze at their current values for ever, and the Router keeps executing swaps under that fixed configuration. On the Hub the *control* tier dies — repointing the other contracts, pausing, removing a bridge coin, de-listing a hook — while a *curator* tier survives that only ever grows the registry: admitting a venue directory (refusing, afterwards, any whose code has moved), adding a bridge coin (add-only, at most three), and listing a hook (refusing one whose code has moved). A malicious listing after renunciation still cannot drain, because pools are validated at quote and at execution and bounded by the floor and your minimum.

What you need not trust.

- *Anyone to price your trade.* The route is derived in the executing frame from public state, or re-measured there if you supplied it.
- *Anyone to route it fairly.* The floor is re-derived on-chain from measured output, and the fee has no term that grows with the gap between quote and delivery.
- *Anyone to count the fee.* The Router refuses to settle without paying it.
- *Anyone to keep serving you.* No routing service, no price feed, no solver network, no keeper, no upgradeable proxy — five zeros where the incumbent design has "required", "common" or "varies". The entry point takes six values a person can assemble by hand, so any script or competitor's front end can call it with no relationship to us of any kind.

THE USUAL DEPENDENCY LIST			THIS ONE		
☒ required	----	something that works out the route	----	☐ none	— the Solver is a view function a
☒ varies	----	a price feed	----	☐ none	
☒ common	----	an auction among outside solvers	----	☐ none	
☒ varies	----	a bot keeping the pool list current	----	☐ none	— vitality decays by arithmetic
☒ common	----	a switch that can replace the code	----	☐ none	— immutable
☒ in practice yes	----	a website	----	☒ a convenience	

each row on the left is something that must keep running for your trade to happen

if every server this project operates disappeared tonight, the contracts would quote, route and settle tomorrow exactly as they did today

LITE. 4 What must be running for your trade to settle: the usual dependency list — a routing service, a price feed, a solver auction, a keeper bot, an upgradeable proxy, a hosted interface — against this one, where the first five are absent and the sixth is a convenience.

And one honest cost, stated as a choice rather than a confession. Immutability cuts both ways: a defect cannot be patched. That is the price of a contract you audit once, and it is why defects here are disclosed and bountied instead of quietly fixed.

12 · Reading the receipt

Every settlement writes a permanent record on-chain. You do not have to take anyone's summary of your own trade.

Take the two-hop route of §4 — A to B by way of WETH — and suppose it settles. The record reads:

FIELD	VALUE	WHAT IT MEANS
<code>user</code>	your address	the payer, not whoever submitted the transaction
<code>tokenOut</code>	B	the token you were buying
<code>quoted</code>	1,000.000 B	the final hop's quote, computed in this same transaction at the WETH that actually arrived — it proves the last hop, not the whole route
<code>realized</code>	998.400 B	what was delivered, measured at the recipient's own balance — 9,984 of 10,000 parts of the quote
<code>floorUsed</code>	933.600 B	the protocol's floor that had to be beaten; the fill sat 64.8 B above it
<code>blockNumber</code>	<i>n</i>	the block, so anyone can re-run the quote at that block and compare

Alongside it, one fee record: `Fee(WETH, 0.014, 0.0042, 0.0098)` — the token the fee was taken in, the amount, and the two treasury shares.

EXECUTION PROOF		
<code>user</code>	0xUSER	 who paid — the payer, not the recipient
<code>tokenOut</code>	B	 the token you receive
<code>quoted</code>	1,000.000	 the last hop's in-frame quote — proves the hop, not the route
<code>realized</code>	998.400	 measured at the recipient after the transfer
<code>floorUsed</code>	933.600	 the floor the contract enforced this time
<code>blockNumber</code>	N	 the block that settled it
<code>Fee(WETH, 0.014, 0.0042, 0.0098)</code>		 the fee, in the token it was actually taken in: 0.014 WETH, split 0.0042 / 0.0098 between the two treasuries
quoted – realized = 1.6 B: the market's answer between quote and fill, above both floors		

LITE. 3 One settlement receipt, annotated: quoted, delivered and floor on one line each, the gap between quoted and delivered named as the market's answer, the fee shown in the token it was actually taken in.

Four readings, and the first is the one people get wrong.

Delivered minus quoted is the market's answer, not the protocol's. You were quoted 1,000.000 B and received 998.400 B, and nothing was taken out of B: the fee left earlier, in WETH, at the second hop's input, and no output-side cut exists on this route at all. The difference between the two figures is what the pools did between the quote and the fill.

Your own minimum was 990 B, so the fill left 8.4 B of slack — 0.84 %. That distance is the number worth watching across many trades: consistently large, and your minimums are looser than they need to be; consistently refused, and they are tighter than the market allows.

floorUsed is what the protocol would not go below. 933.600 B here. It was not your protection — yours was the 990 — but it is the one that would have bound had you set 900.

A fee record denominated in the token you bought appears on exactly one shape of route: a one-hop trade whose destination is itself a bridge coin. On every other shape the fee is taken from an input, earlier, in a coin the treasury holds.

Because every settlement emits one of these, and because the quote it names was produced in the same frame from public state, anyone can re-derive it at that block by a free read of the chain — which is what makes execution quality auditable per transaction rather than per quarter.

13 · How to check any claim yourself

Nothing in this document asks to be believed. Each headline number is reproduced by one command from a clean copy of the repository [24], with the Foundry toolchain and Python installed. Three of them are worth running first, because between them they cover the three things people most reasonably doubt.

1 — Does the test suite actually pass?

```
forge test --no-match-path 'test/fork/**'
```

This compiles the five contracts under the exact settings the release build uses, deploys them into a simulated chain, and runs every test that does not need a real network. Expect **1,319 passed, 0 failed, 1 skipped** of 1,320. The excluded fork lane needs an archive node key; run it with `--match-path 'test/fork/**'` and expect 119 of 119. A different denominator means the tree you checked out is not the one this paper measured — `main` at commit `8949a9d`.

2 — Could those tests have failed?

```
python3 .github/scripts/mutants.py
```

This is the interesting one. For each of the 203 registered mutants, the script edits one exact line of the source, runs the single test that is supposed to notice, requires it to go red, then restores the line. Expect **203 killed of 203**, with any mutation the compiler optimised away reported separately as *inert* rather than counted as a kill. A companion script, `check_targets.py`, verifies that every mutant still points at exactly one line, so the register cannot rot silently while the code moves.

3 — Does a quote survive the world moving?

```
forge test --match-path test/QuoteDelayStatistics.t.sol -vv
```

This runs the 240-sample experiment of §7 and prints the two tables in this document: the drift buckets, the delay buckets, the settle-or-refuse counts and the delivered-over-predicted ratios. `-vv` makes it print the numbers rather than just a pass. The live-Base version is `test/fork/QuoteDelayFork.t.sol` and needs a node key.

The whitepaper's Appendix C lists the rest — the covering arrays, the hostile-venue matrix, the sandwich curve, the canonical oracles, the metamorphic relations, the three-binary agreement, the contract sizes against the 24,576-byte code-size limit set by Ethereum Improvement Proposal 170 [17], and the record of what is deployed on chain today. One line covers them all: every number that comes from the repository has a command beside it, and if a command's output disagrees with this document, the command is right.

Glossary

Aggregator — software that reads many liquidity pools, picks the best combination for your trade, and performs it.

Anchored regime — the fee charged once, on the first bridge coin the Router holds during your route, or on the output of a one-hop route ending in a bridge coin.

Basis point (bps) — one hundredth of one per cent. 100 bps = 1 %.

Believability band — the $\pm 5\%$ window around the depth-weighted middle of the candidates' prices; a pool quoting outside it is dropped before anything is sent.

Blockchain — a public ledger kept by many independent computers that agree on its contents.

Bridge coin — one of at most three widely-held tokens the protocol routes through, and the coin the fee is taken in on an anchored route.

Calldata — the data sent with a transaction; supplying your own route in it is one of the four ways in.

Concentrated liquidity — a pool that stores a price and an active depth instead of two stocks, concentrating its capital near the current price.

Constant product — the oldest pool rule: the two stocks multiplied together may never shrink.

Core, Hub, Solver, Router, Quoter — the five contracts: mathematics; memory of pools; route choice; the only mover of money; the preview.

Deterministic Derivation $\mathcal{D}$ — computing a pool's address from public inputs instead of fetching it, and proving what it cannot compute.

Drift — how far the pools moved between your quote and your transaction's inclusion.

Eightfold Dispatcher Ω — the one function that prices every kind of pool; six kinds live today across four mathematical families.

Exhaustion regime — a route with no bridge coin as any hop's input: every hop pays 28 bps of its own measured input, which is what makes a worthless prefix useless as a fee dodge.

Fee-on-transfer token — a token that delivers less than it was sent; every amount that matters here is a measured balance change rather than an assumed one.

Front end — the website or app that proposes a trade to you; a convenience here, not a dependency.

Gas — the fee you pay the network for the computation your transaction performs.

Hook — code a Uniswap V4 pool runs in the middle of a swap. The protocol never asks a hook anything; it refuses the two permissions that would let one alter the accounting, by reading bits fixed in the hook's own address.

Hop — one stage of a route, from one token to the next. At most three.

Iron-Law floor Φ — the protocol's own retention floor, re-derived inside the executing transaction: 96 % as a base, loosening with the impact the trade really caused and with how genuinely it was split, never below 80 %.

Leg — one pool inside one hop; a hop can be split across several.

Liquidity pool — a contract holding a stock of two tokens and willing to swap one for the other at a price its own arithmetic decides.

Meta-Equation — the whole aggregator as one maximisation, written as a product so that a single failing factor makes a route worth exactly zero.

Metrological Design — build the contract as an instrument: measure every quantity that can be measured, and shape the rest so a false value can only hurt whoever supplied it.

Minimum output — the number you set, below which your trade must not fill. Mandatory; a zero minimum is refused before a token moves.

Monoslot — a pool's entire routing state in one 256-bit word. It contains no price.

Mutant / mutation guard — a deliberate one-line break in the source, paired with the single test that must go red because of it. 203 registered, 203 killed.

One-Way Door — surrendering administrative control irreversibly in code: the control tier dies for ever, and only a grow-only curator tier remains.

Preview — the advisory quote a read-only call returns before you sign; the binding numbers are the ones the Router re-derives in the executing frame.

Route — the whole path from your token to the one you want: hops, and legs inside hops.

Sandwich — trading ahead of a pending transaction, letting it fill worse, and trading back.

Self-Healing Registry — the protocol learns which pools exist by trading through them: sixteen seats per pair, fitness-ranked eviction, activity that decays by arithmetic rather than by a keeper.

Slippage — the price movement your own trade causes as it goes through a pool.

Smart contract — a program stored on the blockchain that anyone can read and call.

Surplus Rule — the take is 28 bps of one measured amount, and no term in settlement scales with the gap between quote and delivery.

Third way — any outcome that is neither a correct settlement inside the floors with nothing left behind, nor a refusal with a code belonging to this protocol. Zero occurred in either covering array.

Token — a ledger entry recording that an address owns a quantity of something.

Vitality Field Ψ — a pool's earned quality score, computed from the Monoslot alone; it ranks and truncates the candidate list, can hide a venue, and can never misprice one.

Wallet — the app holding the keys that authorise your transactions.

References

Numbering follows the technical whitepaper's reference list; only the entries cited in this document appear here.

1. G. Angeris, A. Evans, T. Chitra, S. Boyd. *Optimal Routing for Constant Function Market Makers*. arXiv:2204.05238, 2022 (ACM EC '22).
2. H. Adams, N. Zinsmeister, D. Robinson. *Uniswap v2 Core*. 2020. <https://uniswap.org/whitepaper.pdf>
3. H. Adams, N. Zinsmeister, M. Salem, R. Keefer, D. Robinson. *Uniswap v3 Core*. 2021. <https://uniswap.org/whitepaper-v3.pdf>
4. Uniswap Labs (H. Adams et al.). *Uniswap v4 Core*. 2024. <https://uniswap.org/whitepaper-v4.pdf>
5. P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, A. Juels. *Flash Boys 2.0: Front-running in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability*. IEEE S&P 2020. DOI 10.1109/SP40000.2020.00040.
6. L. Zhou, K. Qin, C. F. Torres, D. V. Le, A. Gervais. *High-Frequency Trading on Decentralized On-Chain Exchanges*. IEEE S&P 2021. DOI 10.1109/SP40001.2021.00027.
7. T. Y. Chen, S. C. Cheung, S. M. Yiu. *Metamorphic Testing: A New Approach for Generating Next Test Cases*. HKUST-CS98-01, 1998; arXiv:2002.12543.
8. R. A. DeMillo, R. J. Lipton, F. G. Sayward. *Hints on Test Data Selection: Help for the Practicing Programmer*. IEEE Computer 11(4), 1978. DOI 10.1109/C-M.1978.218136.
9. A. Avizienis. *The N-Version Approach to Fault-Tolerant Software*. IEEE TSE SE-11(12), 1985. DOI 10.1109/TSE.1985.231893.
10. D. R. Kuhn, R. N. Kacker, Y. Lei. *Practical Combinatorial Testing*. NIST SP 800-142, 2010. DOI 10.6028/NIST.SP.800-142.
11. B. Littlewood, L. Strigini. *Validation of Ultrahigh Dependability for Software-Based Systems*. Communications of the ACM 36(11), 1993. DOI 10.1145/163359.163373.
12. V. Buterin. *EIP-170: Contract Code Size Limit*. <https://eips.ethereum.org/EIPS/eip-170>

13. M. Lundfall et al. *EIP-2612: Permit Extension for EIP-20 Signed Approvals*. <https://eips.ethereum.org/EIPS/eip-2612> · Uniswap Labs. *Permit2*. <https://github.com/Uniswap/permit2>
14. MariaDB Corporation Ab. *Business Source License 1.1*. <https://mariadb.com/bsl11/>
15. Mitra. *BlazePhoenix Staking Engine — Design Specification, Version 1.0*. <https://blazephoenix.xyz/staking-whitepaper.md>
16. Fable & Mitra. *BlazePhoenix-Dex: an on-chain DEX aggregator with measured routing*. Repository, `main` 8949a9d, 2026-09-05. <https://github.com/blazephoenixxyz-crypto/Blaze-Phoenix-Dex>